Beyond GLMs with splines

Bert van der Veen

University of Bayreuth

Outline Today

- Ridge regression
- Generalized Additive Models
- Penalized splines
- Smoothing parameter selection
- GAMs as GLMMs
- Example
- 15 minute break
- 1 Lecture/presentation
- 1 Practical

Questions about yesterday?

Resources

- **Fieberg, J.** (2024). *Statistics for Ecologists.* https://statistics4ecologists-v3.netlify.app
  Ch. 4: Non-linear models — intuitive treatment of splines and basis functions
- **Simpson, G.** Physalia GAM course. https://github.com/gavinsimpson/physalia-gam-course
  Comprehensive practical course with `mgcv`; slides and code freely available
- **Wood, S.N.** (2017). *Generalized Additive Models: An Introduction with R*, 2nd ed. Chapman & Hall.
  The definitive reference; covers all theory and `mgcv` in depth

R packages for GAMs

Fitting

- `mgcv` — the standard; most features, best documented
- `gamm4` — GAMMs via `lme4` backend (Gaussian)
- `glmmTMB` — GAM smooths + arbitrary GLMM structure

Visualisation & diagnostics

- `gratia` — `ggplot2`-based smooth plots, `draw()`, `appraise()`
- `DHARMa` — simulation residuals (works with `mgcv`)
- `ggeffects` — marginal predictions for `mgcv` models

We will use `mgcv` for fitting and `gratia` for visualisation

Where we are

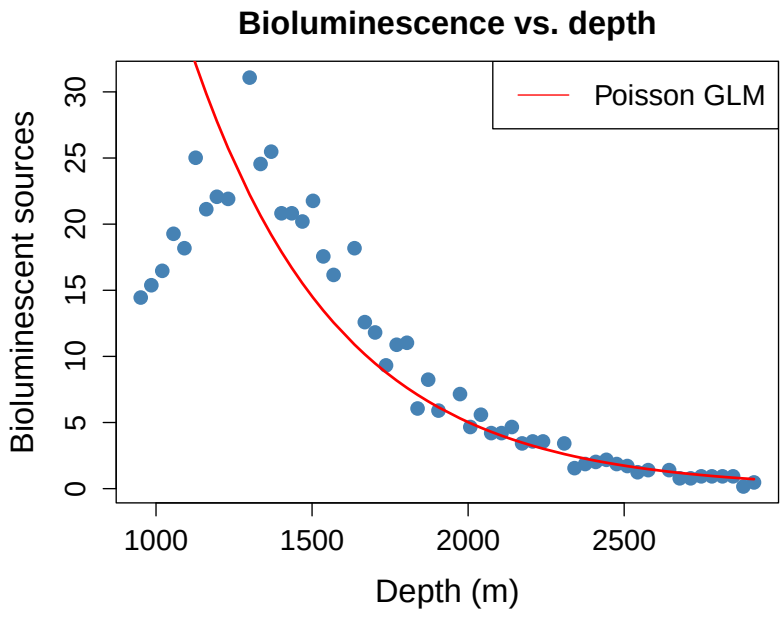
Model	**Non-linear covariates**	**Correlation**
GLM	No	No
GLMM	No	Yes (random effects)
GAM	Yes (smooth functions)	Optional

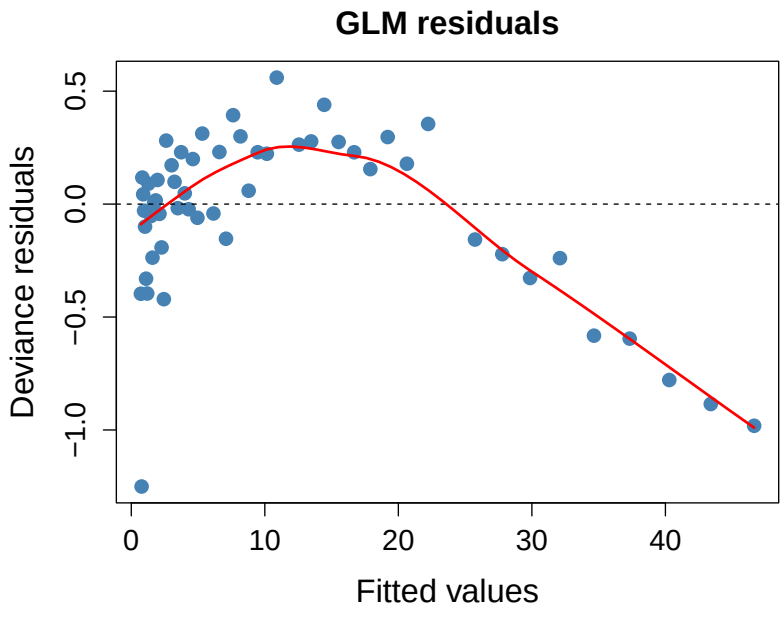
Where we are

Model	Non-linear covariates	Correlation
GLM	No	No
GLMM	No	Yes (random effects)
GAM	Yes (smooth functions)	Optional

- GLMs and GLMMs do have non-linearity through the link function, but assume covariates enter the linear predictor **linearly**
- Many ecological relationships are non-linear

Non-linearity in ecology





A Poisson GLM misses the non-linear relationship.

Penalized regression

Ordinary least squares minimizes:

$$\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 = \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2 \tag{1}$$

Penalized regression

Ordinary least squares minimizes:

$$\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 = \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2 \tag{1}$$

Ridge regression (Hoerl and Kennard 1970) adds a penalty on the size of the coefficients:

$$\|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2 + \lambda\|\boldsymbol{\beta}\|^2 \tag{2}$$

- $\lambda \geq 0$ controls how strongly coefficients are penalized towards zero
- $\lambda = 0$: ordinary least squares
- $\lambda \to \infty$: all $\widehat{\beta}_j \to 0$

Ridge regression: solution

The ridge estimator has a closed form:

$$\hat{\beta}_{\text{ridge}} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y} \tag{3}$$

Ridge regression: solution

The ridge estimator has a closed form:

$$\hat{\beta}_{\text{ridge}} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y} \tag{3}$$

- The $\lambda \mathbf{I}$ term regularizes the problem — helps when $\mathbf{X}^\top \mathbf{X}$ is ill-conditioned
- Ridge regression **shrinks** estimates toward zero
- Very useful in high-dimensional settings (more predictors than observations)

Ridge regression as a GLMM

Ridge regression is equivalent to a mixed model with:

$$\boldsymbol{\beta} \sim \mathcal{N}\left(\mathbf{0}, \frac{\sigma^2}{\lambda}\mathbf{I}\right) \tag{4}$$

Ridge regression as a GLMM

Ridge regression is equivalent to a mixed model with:

$$\boldsymbol{\beta} \sim \mathcal{N}\left(\mathbf{0}, \frac{\sigma^2}{\lambda}\mathbf{I}\right) \tag{4}$$

For a Gaussian model with **known** λ: minimizing the penalized sum of squares gives the posterior mode (MAP estimate) of $\boldsymbol{\beta}$ — the conditional mode given the data.

Ridge regression as a GLMM

Ridge regression is equivalent to a mixed model with:

$$\boldsymbol{\beta} \sim \mathcal{N}\left(\mathbf{0}, \frac{\sigma^2}{\lambda}\mathbf{I}\right) \tag{4}$$

For a Gaussian model with **known** λ: minimizing the penalized sum of squares gives the posterior mode (MAP estimate) of $\boldsymbol{\beta}$ — the conditional mode given the data.

The penalty λ controls variance: small $\lambda \Leftrightarrow$ large variance $\Leftrightarrow$ flexible

Ridge regression as a GLMM

Ridge regression is equivalent to a mixed model with:

$$\boldsymbol{\beta} \sim \mathcal{N}\left(\mathbf{0}, \frac{\sigma^2}{\lambda}\mathbf{I}\right) \tag{4}$$

For a Gaussian model with **known** λ: minimizing the penalized sum of squares gives the posterior mode (MAP estimate) of $\boldsymbol{\beta}$ — the conditional mode given the data.

The penalty λ controls variance: small $\lambda \Leftrightarrow$ large variance $\Leftrightarrow$ flexible

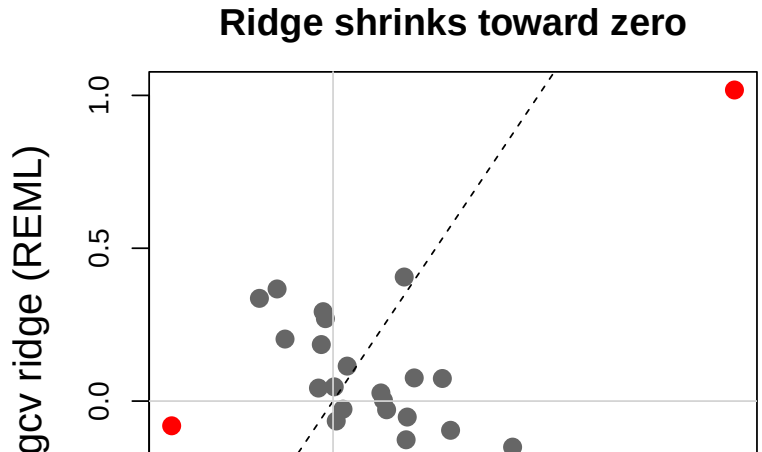
GAMs extend this idea: instead of penalizing the size of $\boldsymbol{\beta}$, we penalize the **wiggliness** of a smooth function.

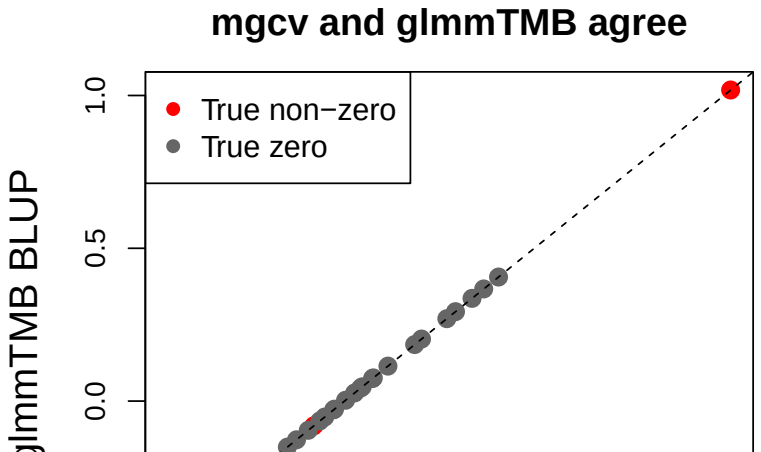
Ridge via mgcv: a code example

```r
library(mgcv)
set.seed(1); n <- 60; p <- 25
X <- matrix(rnorm(n*p), n, p)
beta_true <- c(2, -1, rep(0, p-2))
y <- 1 + X %*% beta_true + rnorm(n, sd=1.5)

# OLS: unstable when p close to n
colnames(X) <- paste0("X", seq_len(p))
dat_r <- data.frame(y=y, X)
m_ols <- lm(y ~ ., data=dat_r)

# Ridge via mgcv: "re" smooth treats each predictor as
# a random slope - equivalent to ridge penalization
dat_long <- data.frame(
```

Ridge via mgcv: comparison

```
# glmmTMB gives the same result: random slopes = ridge
library(glmmTMB)
m_glmm <- glmmTMB(y ~ (0+xval|pred), data=dat_long)

b_ols  <- coef(m_ols)[-1]
b_mgcv <- coef(m_mgcv)[-1]          # BLUP from mgcv
b_glmm <- ranef(m_glmm)$cond$pred[,"xval"]
```





The distinction does not quite hold for GLMMs

- **Ridge**: λ is fixed externally (cross-validation, AIC); β is optimized directly — no marginal likelihood

The distinction does not quite hold for GLMMs

- **Ridge**: λ is fixed externally (cross-validation, AIC); $\boldsymbol{\beta}$ is optimized directly — no marginal likelihood
- **GLMM**: variance components (including the ratio $\sigma^2/\sigma_b^2 = \lambda$) are estimated by **marginalizing over** the random effects via ML or REML; the BLUP is then computed conditionally on those estimates

The distinction does not quite hold for GLMMs

- **Ridge**: λ is fixed externally (cross-validation, AIC); $\boldsymbol{\beta}$ is optimized directly — no marginal likelihood
- **GLMM**: variance components (including the ratio $\sigma^2/\sigma_b^2 = \lambda$) are estimated by **marginalizing over** the random effects via ML or REML; the BLUP is then computed conditionally on those estimates
- **Non-Gaussian GLMMs**: the posterior $p(\mathbf{u}|\mathbf{y})$ is not Gaussian, so posterior mode $\neq$ posterior mean; the BLUP (posterior mean) requires integration, not just optimization

The distinction does not quite hold for GLMMs

- **Ridge**: λ is fixed externally (cross-validation, AIC); $\boldsymbol{\beta}$ is optimized directly — no marginal likelihood
- **GLMM**: variance components (including the ratio $\sigma^2/\sigma_b^2 = \lambda$) are estimated by **marginalizing over** the random effects via ML or REML; the BLUP is then computed conditionally on those estimates
- **Non-Gaussian GLMMs**: the posterior $p(\mathbf{u}|\mathbf{y})$ is not Gaussian, so posterior mode $\neq$ posterior mean; the BLUP (posterior mean) requires integration, not just optimization
- The equivalence holds **exactly** only for Gaussian models with known variance components

The distinction does not quite hold for GLMMs

- **Ridge**: λ is fixed externally (cross-validation, AIC); $\boldsymbol{\beta}$ is optimized directly — no marginal likelihood
- **GLMM**: variance components (including the ratio $\sigma^2/\sigma_b^2 = \lambda$) are estimated by **marginalizing over** the random effects via ML or REML; the BLUP is then computed conditionally on those estimates
- **Non-Gaussian GLMMs**: the posterior $p(\mathbf{u}|\mathbf{y})$ is not Gaussian, so posterior mode $\neq$ posterior mean; the BLUP (posterior mean) requires integration, not just optimization
- The equivalence holds **exactly** only for Gaussian models with known variance components

The analogy is still useful for intuition, but the mechanics differ

GAM formulation

A Generalized Additive Model replaces linear predictors with smooth functions:

$$g\{\mathsf{E}(y_i|\mathbf{x}_i)\} = \alpha + f_1(x_{i1}) + f_2(x_{i2}) + \cdots \tag{5}$$

GLM:

$$g\{\mathsf{E}(y_i)\} = \alpha + x_{i1}\beta_1 + x_{i2}\beta_2$$

GAM:

$$g\{\mathsf{E}(y_i)\} = \alpha + f_1(x_{i1}) + f_2(x_{i2})$$

GAM formulation

A Generalized Additive Model replaces linear predictors with smooth functions:

$$g\{\mathsf{E}(y_i|\mathbf{x}_i)\} = \alpha + f_1(x_{i1}) + f_2(x_{i2}) + \cdots \tag{5}$$

GLM:

$$g\{\mathsf{E}(y_i)\} = \alpha + x_{i1}\beta_1 + x_{i2}\beta_2$$

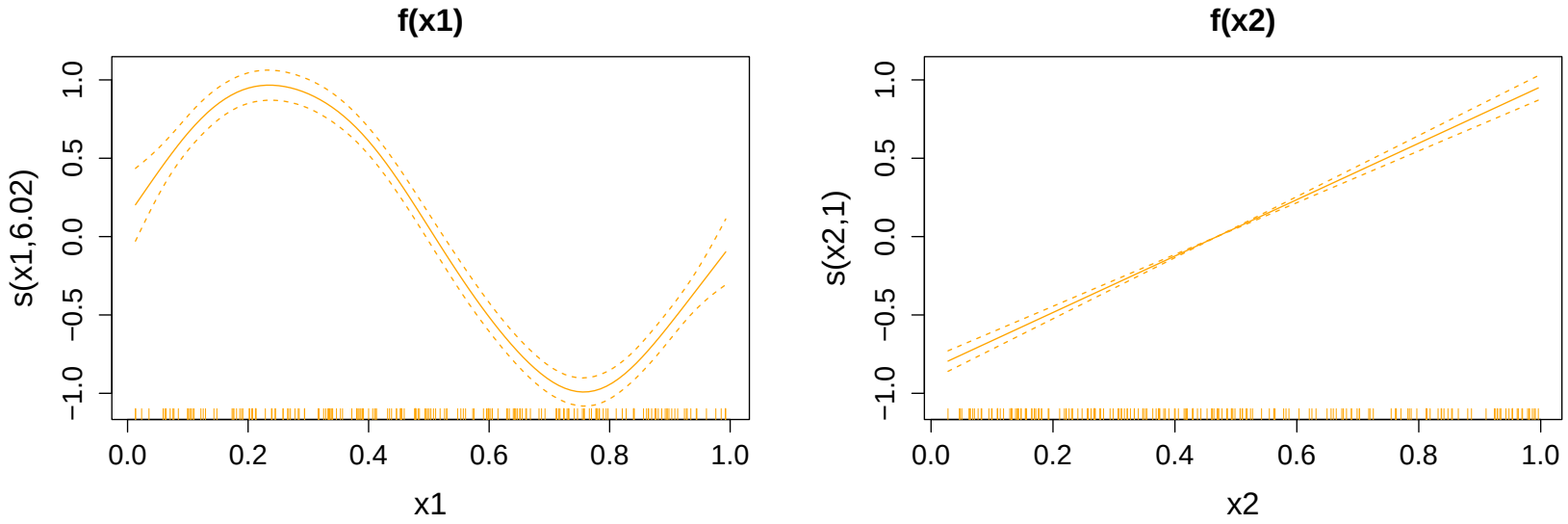
GAM:

$$g\{\mathsf{E}(y_i)\} = \alpha + f_1(x_{i1}) + f_2(x_{i2})$$

- $f_j(\cdot)$ are smooth, flexible functions estimated from data
- Linear terms $x\beta$ are a special case
- Random effects can be included:
  $g\{\mathsf{E}(y_i|u_j)\} = \alpha + f(x_i) + u_j$

Why "additive"?

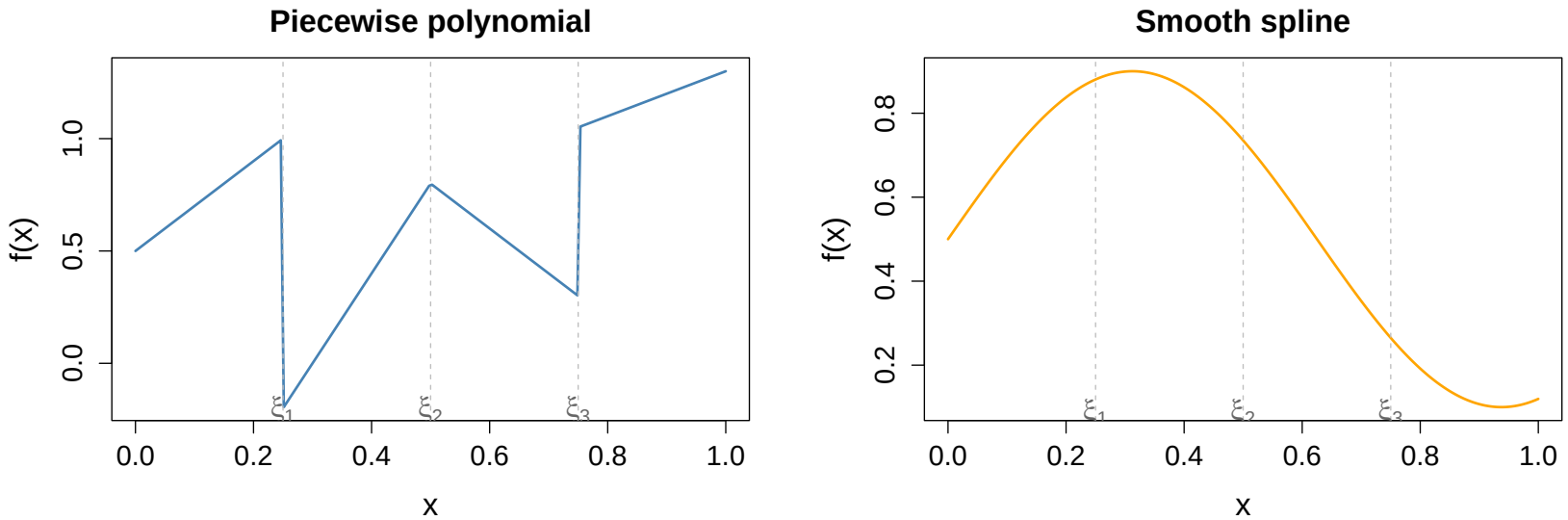
Each smooth contributes **additively** to the linear predictor — just as in a GLM.



Each smooth is estimated **while holding all others constant** — the partial effect.

What is a spline?

A spline is a **piecewise polynomial** — multiple polynomial segments joined smoothly at **knots** $\xi_1, \ldots, \xi_K$



A smooth spline constrains adjacent segments to join with **continuous derivatives** — no kinks.

Basis functions: building blocks

Any smooth $f(x)$ can be represented as a **weighted sum of basis functions** $b_j(x)$:

$$f(x) = \sum_{j=1}^{K} \beta_j \, b_j(x) = \mathbf{b}(x)^\top \boldsymbol{\beta} \tag{6}$$

Basis functions: building blocks

Any smooth $f(x)$ can be represented as a **weighted sum of basis functions** $b_j(x)$:

$$f(x) = \sum_{j=1}^{K} \beta_j \, b_j(x) = \mathbf{b}(x)^\top \boldsymbol{\beta} \tag{6}$$

This is the same $\mathbf{X}\boldsymbol{\beta}$ structure as a GLM — we simply **redefine the design matrix X** to contain basis function columns:

$$\mathbf{X}\boldsymbol{\beta}, \quad \text{where } X_{ij} = b_j(x_i) \tag{7}$$

Basis functions: building blocks

Any smooth $f(x)$ can be represented as a **weighted sum of basis functions** $b_j(x)$:

$$f(x) = \sum_{j=1}^{K} \beta_j \, b_j(x) = \mathbf{b}(x)^\top \boldsymbol{\beta} \tag{6}$$

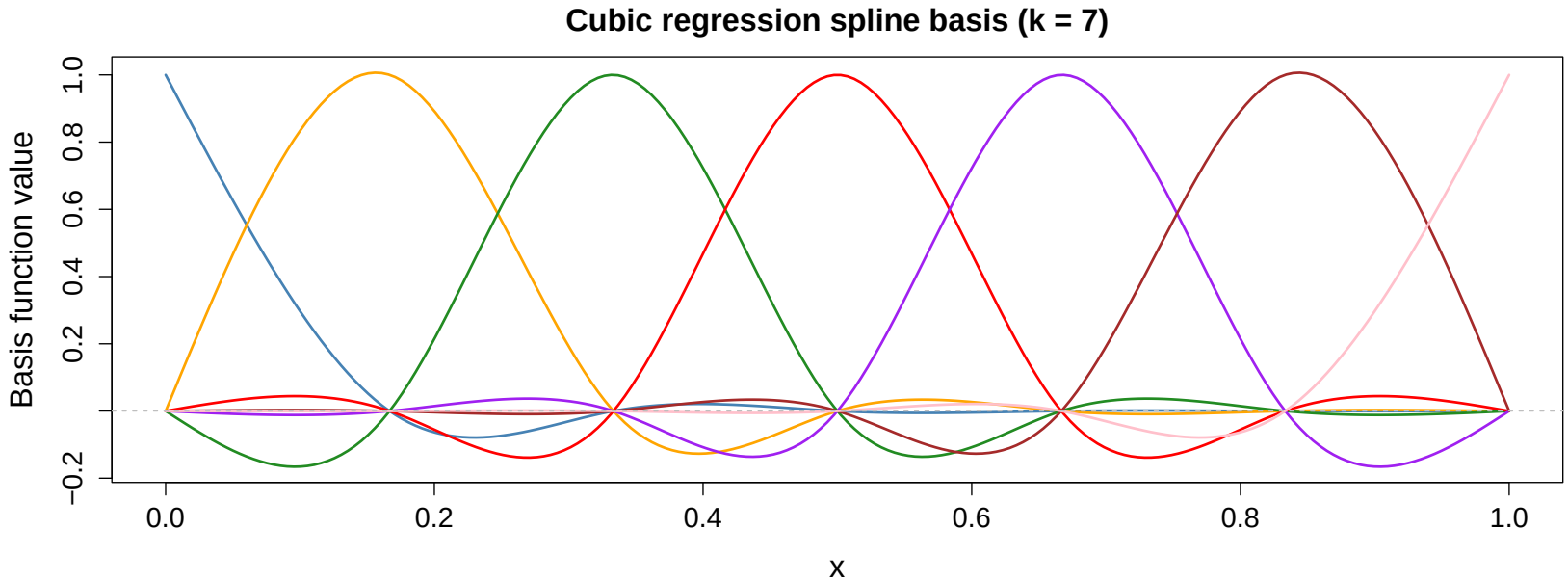
This is the same $\mathbf{X}\boldsymbol{\beta}$ structure as a GLM — we simply **redefine the design matrix X** to contain basis function columns:

$$\mathbf{X}\boldsymbol{\beta}, \quad \text{where } X_{ij} = b_j(x_i) \tag{7}$$

- No new estimation machinery needed
- Penalization is the only addition
- The same link functions and distributions apply

Cubic regression splines

The cubic regression spline (CRS) basis uses cubic polynomials between knots, constrained to have continuous first and second derivatives at knots.



Each basis function has **local support** — it is non-zero only near its knot.

Thin plate regression splines

The **default in `mgcv`** (Wood 2003) . A TPS finds the function f minimising:

$$\sum_{i=1}^{n}\{y_i - f(x_i)\}^2 + \lambda \int \{f''(x)\}^2\, dx \tag{8}$$

over **all** smooth functions — no knots to choose. The solution is a natural spline with knots at every data point. For large n, `mgcv` uses a low-rank approximation: keep only the k leading eigenfunctions of the penalty operator.

Thin plate regression splines

The **default in `mgcv`** (Wood 2003) . A TPS finds the function f minimising:

$$\sum_{i=1}^{n}\{y_i - f(x_i)\}^2 + \lambda \int \{f''(x)\}^2\, dx \tag{8}$$

over **all** smooth functions — no knots to choose. The solution is a natural spline with knots at every data point. For large n, `mgcv` uses a low-rank approximation: keep only the k leading eigenfunctions of the penalty operator.

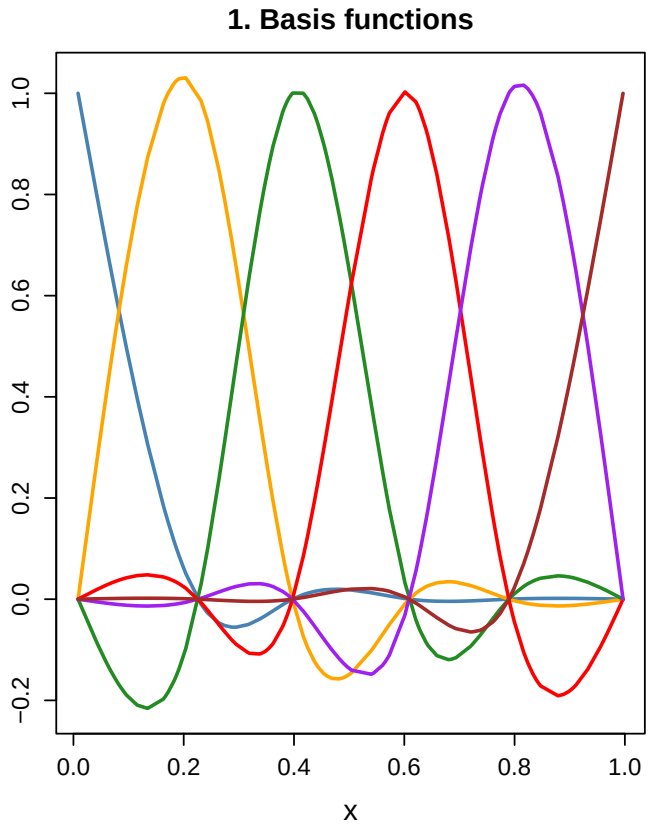
P-splines (Eilers and Marx 1996): B-spline basis + difference penalty on adjacent coefficients

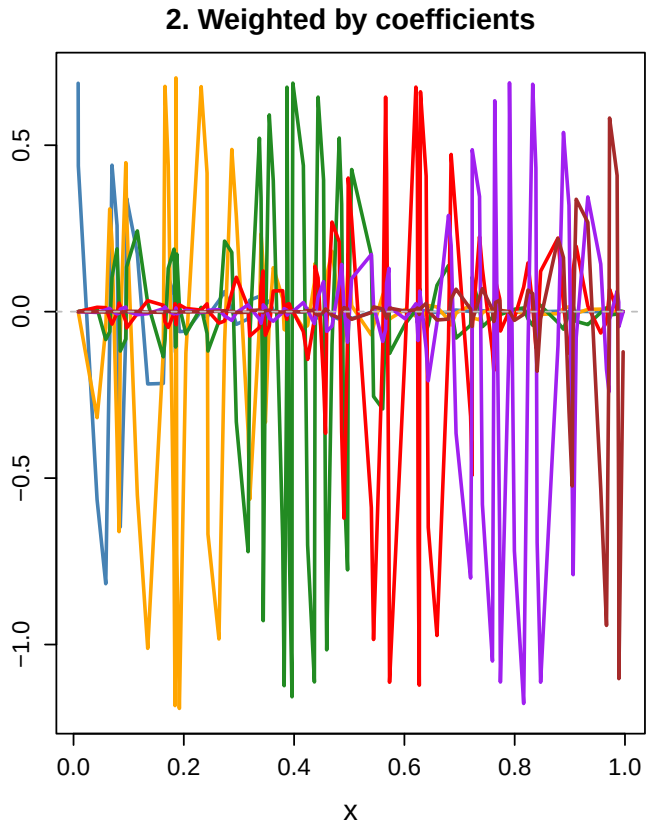
Cyclic cubic splines (`bs="cc"`): smooth joins at endpoints — for periodic predictors (month, day of year)

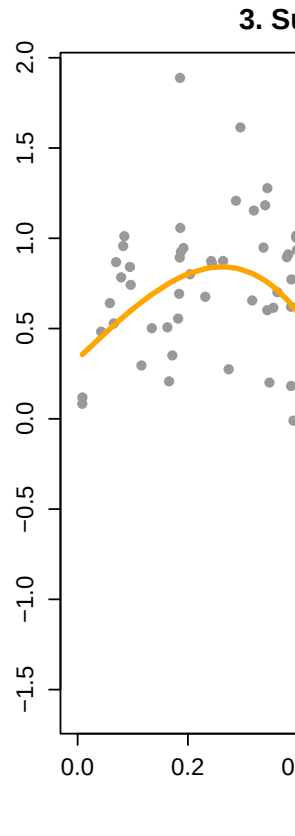
Spline summary

Type	**mgcv code**	**When to use**
Thin plate (default)	`s(x)`	General purpose
Cubic regression	`s(x, bs="cr")`	Interpretable, fast
P-spline	`s(x, bs="ps")`	Large datasets
Cyclic cubic	`s(x, bs="cc")`	Periodic predictors
Duchon spline	`s(x, bs="ds")`	Spatial data

Building a smooth: step by step







Wiggliness penalty

Without a penalty, enough basis functions can fit *any* dataset perfectly (overfitting).

Wiggliness penalty

Without a penalty, enough basis functions can fit *any* dataset perfectly (overfitting).

The **wiggliness penalty** discourages overly complex smooths:

$$\text{PRSS} = \underbrace{\sum_{i=1}^{n} \{y_i - f(x_i)\}^2}_{\text{Fit to data}} + \lambda \underbrace{\int \{f''(x)\}^2 \, dx}_{\text{Wiggliness}} \tag{9}$$

Wiggliness penalty

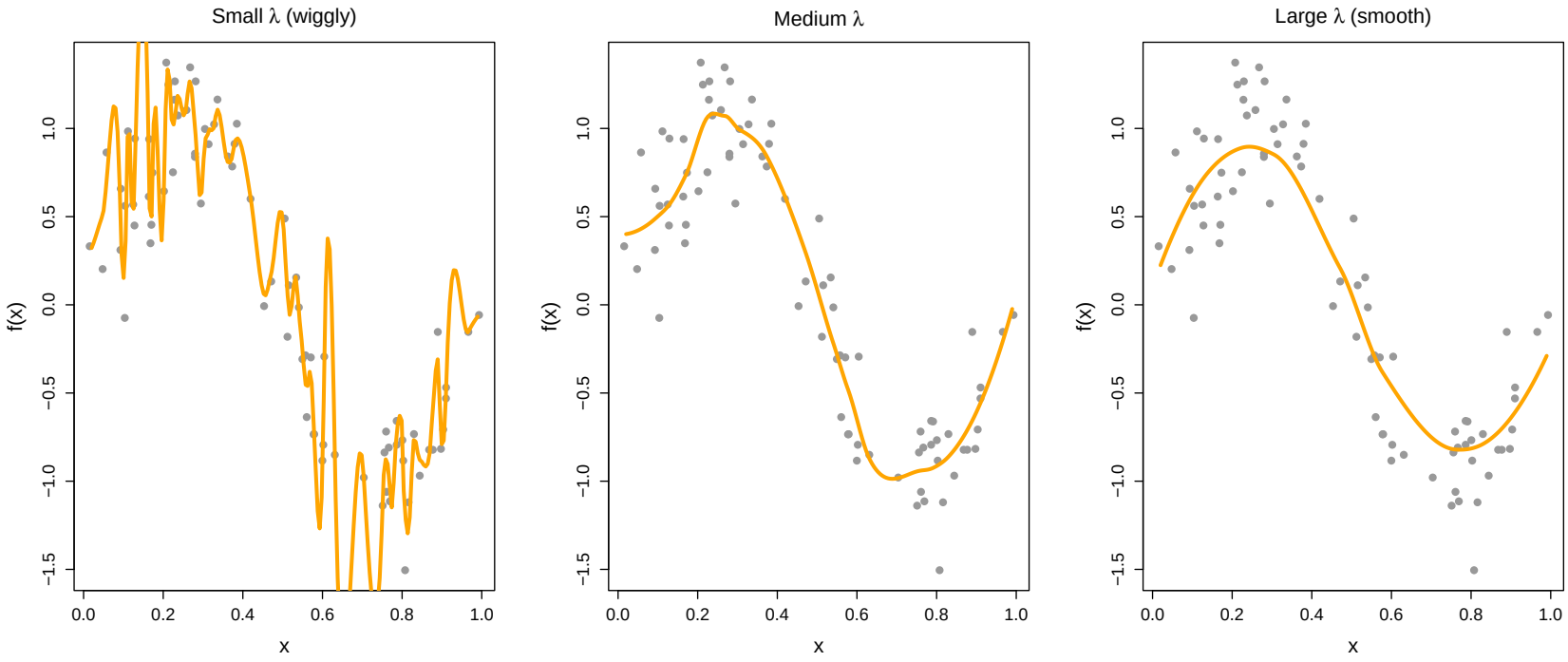
Without a penalty, enough basis functions can fit *any* dataset perfectly (overfitting).

The **wiggliness penalty** discourages overly complex smooths:

$$\text{PRSS} = \underbrace{\sum_{i=1}^{n} \{y_i - f(x_i)\}^2}_{\text{Fit to data}} + \lambda \underbrace{\int \{f''(x)\}^2 \, dx}_{\text{Wiggliness}} \tag{9}$$

- $f''(x)$ is the second derivative — measures curvature
- $\lambda \geq 0$ is the **smoothing parameter**
- $\lambda = 0$: interpolating spline (overfits)
- $\lambda \to \infty$: straight line (linear regression)

Smoothing parameter λ



The same data; only λ differs.

The penalty in matrix form

For a penalized spline with basis matrix $\mathbf{B}$ and coefficients $\boldsymbol{\beta}$:

$$\text{PRSS} = \|\mathbf{y} - \mathbf{B}\boldsymbol{\beta}\|^2 + \lambda\, \boldsymbol{\beta}^\top \mathbf{S}\, \boldsymbol{\beta} \tag{10}$$

- $\mathbf{S}$ is the **penalty matrix** encoding the wiggliness constraint
- For a cubic spline: $S_{jk} = \int b_j''(x)\, b_k''(x)\, dx$

The penalty in matrix form

For a penalized spline with basis matrix $\mathbf{B}$ and coefficients $\boldsymbol{\beta}$:

$$\text{PRSS} = \|\mathbf{y} - \mathbf{B}\boldsymbol{\beta}\|^2 + \lambda\, \boldsymbol{\beta}^\top \mathbf{S}\, \boldsymbol{\beta} \tag{10}$$

- $\mathbf{S}$ is the **penalty matrix** encoding the wiggliness constraint
- For a cubic spline: $S_{jk} = \int b_j''(x)\, b_k''(x)\, dx$

The solution is:

$$\hat{\boldsymbol{\beta}} = (\mathbf{B}^\top \mathbf{B} + \lambda \mathbf{S})^{-1} \mathbf{B}^\top \mathbf{y} \tag{11}$$

The penalty in matrix form

For a penalized spline with basis matrix $\mathbf{B}$ and coefficients $\boldsymbol{\beta}$:

$$\text{PRSS} = \|\mathbf{y} - \mathbf{B}\boldsymbol{\beta}\|^2 + \lambda\, \boldsymbol{\beta}^\top \mathbf{S}\, \boldsymbol{\beta} \tag{10}$$

- $\mathbf{S}$ is the **penalty matrix** encoding the wiggliness constraint
- For a cubic spline: $S_{jk} = \int b_j''(x)\, b_k''(x)\, dx$

The solution is:

$$\hat{\boldsymbol{\beta}} = (\mathbf{B}^\top \mathbf{B} + \lambda \mathbf{S})^{-1} \mathbf{B}^\top \mathbf{y} \tag{11}$$

Compare to ridge: $(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$ — same structure, $\mathbf{S}$ replaces $\mathbf{I}$

Effective degrees of freedom

The effective degrees of freedom (EDF) of a smooth quantifies its complexity:

$$\text{EDF} = \text{tr}\left\{(\mathbf{B}^\top\mathbf{B} + \lambda\mathbf{S})^{-1}\mathbf{B}^\top\mathbf{B}\right\} \tag{12}$$

- EDF $= 1$: straight line
- EDF $= 2$: quadratic
- EDF $\approx k$: nearly interpolating
- Reported by `summary.gam`
- Used for hypothesis tests
- Larger EDF $\Rightarrow$ more complex smooth

How do we choose λ?

Several criteria exist for selecting the smoothing parameter automatically:

1. **GCV** — Generalized Cross-Validation
2. **ML** — Maximum Likelihood
3. **REML** — Restricted Maximum Likelihood

All treat λ as something to be estimated from the data.

GCV: Generalized Cross-Validation

Proposed by Craven and Wahba (1979), popularized for GAMs by **Wood (2004)**:

$$\text{GCV}(\lambda) = \frac{n \cdot \text{RSS}(\lambda)}{\{n - \text{EDF}(\lambda)\}^2} \tag{13}$$

GCV: Generalized Cross-Validation

Proposed by Craven and Wahba (1979), popularized for GAMs by **Wood (2004)**:

$$\text{GCV}(\lambda) = \frac{n \cdot \text{RSS}(\lambda)}{\{n - \text{EDF}(\lambda)\}^2} \tag{13}$$

- Approximates leave-one-out cross-validation at $O(n)$ cost
- Penalizes model complexity via EDF
- Minimizing GCV over λ balances fit vs. smoothness

GCV: Generalized Cross-Validation

Proposed by Craven and Wahba (1979), popularized for GAMs by **Wood (2004)**:

$$\text{GCV}(\lambda) = \frac{n \cdot \text{RSS}(\lambda)}{\{n - \text{EDF}(\lambda)\}^2} \tag{13}$$

- Approximates leave-one-out cross-validation at $O(n)$ cost
- Penalizes model complexity via EDF
- Minimizing GCV over λ balances fit vs. smoothness

Problem: GCV can undersmooth (too wiggly) in some cases, especially with correlated errors (Reiss and Ogden 2009)

ML and REML

The GLMM representation of the smooth (see next section) enables likelihood-based selection:

ML and REML

The GLMM representation of the smooth (see next section) enables likelihood-based selection:

ML (Maximum Likelihood): maximize the marginal likelihood of **y** over both $\boldsymbol{\beta}$ and λ

ML and REML

The GLMM representation of the smooth (see next section) enables likelihood-based selection:

ML (Maximum Likelihood): maximize the marginal likelihood of **y** over both β and λ

REML (Restricted Maximum Likelihood): maximize the likelihood after projecting out fixed effects

- REML is less prone to undersmoothing than ML (Wood 2011)
- REML accounts for uncertainty in fixed effects when estimating λ
- Generally recommended; default in `mgcv` since version 1.8

ML and REML

The GLMM representation of the smooth (see next section) enables likelihood-based selection:

ML (Maximum Likelihood): maximize the marginal likelihood of **y** over both β and λ

REML (Restricted Maximum Likelihood): maximize the likelihood after projecting out fixed effects

- REML is less prone to undersmoothing than ML (Wood 2011)
- REML accounts for uncertainty in fixed effects when estimating λ
- Generally recommended; default in `mgcv` since version 1.8

Use `method="REML"` in `mgcv::gam()` (it is the default)

Comparing criteria

Criterion	**Basis**	**Tends to**	**Use when**
GCV	CV approximation	Undersmooth	Legacy, simple models
ML	Likelihood	Undersmooth	Rarely preferred
REML	Restricted likelihood	Best calibrated	Default (recommended)

Comparing criteria

Criterion	Basis	Tends to	Use when
GCV	CV approximation	Undersmooth	Legacy, simple models
ML	Likelihood	Undersmooth	Rarely preferred
REML	Restricted likelihood	Best calibrated	Default (recommended)

In R:

```
gam(y ~ s(x), method = "REML")   # default, recommended
gam(y ~ s(x), method = "GCV.Cp") # GCV
gam(y ~ s(x), method = "ML")     # ML
```

The GLMM representation

(Wood 2004, 2011)

The penalty matrix **S** is rank-deficient: it has a **null space** (directions that are not penalized).

The GLMM representation

(Wood 2004, 2011)

The penalty matrix $\mathbf{S}$ is rank-deficient: it has a **null space** (directions that are not penalized).

Decompose $\mathbf{S}$ via spectral decomposition $\mathbf{S} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top$ and partition the smooth:

$$f(x) = \underbrace{\mathbf{B}_0\boldsymbol{\beta}_0}_{\text{Unpenalized (fixed)}} + \underbrace{\mathbf{B}_1\mathbf{b}}_{\text{Penalized (random)}} \tag{14}$$

The GLMM representation

(Wood 2004, 2011)

The penalty matrix $\mathbf{S}$ is rank-deficient: it has a **null space** (directions that are not penalized).

Decompose $\mathbf{S}$ via spectral decomposition $\mathbf{S} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top$ and partition the smooth:

$$f(x) = \underbrace{\mathbf{B}_0\boldsymbol{\beta}_0}_{\text{Unpenalized (fixed)}} + \underbrace{\mathbf{B}_1\mathbf{b}}_{\text{Penalized (random)}} \tag{14}$$

with $\mathbf{b} \sim \mathcal{N}(\mathbf{0}, \frac{\sigma^2}{\lambda}\mathbf{I})$

The GLMM representation

(Wood 2004, 2011)

The penalty matrix $\mathbf{S}$ is rank-deficient: it has a **null space** (directions that are not penalized).

Decompose $\mathbf{S}$ via spectral decomposition $\mathbf{S} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top$ and partition the smooth:

$$f(x) = \underbrace{\mathbf{B}_0\boldsymbol{\beta}_0}_{\text{Unpenalized (fixed)}} + \underbrace{\mathbf{B}_1\mathbf{b}}_{\text{Penalized (random)}} \tag{14}$$

with $\mathbf{b} \sim \mathcal{N}(\mathbf{0}, \frac{\sigma^2}{\lambda}\mathbf{I})$

This is **exactly** a GLMM: fixed-effects design matrix $\mathbf{X} = \mathbf{B}_0$ and random-effects design matrix $\mathbf{Z} = \mathbf{B}_1$.

What this means in practice

- GAMs can be fitted using GLMM software
- The smoothing parameter λ is determined by the random-effect variance: $\sigma_b^2 = \sigma^2/\lambda$
- REML estimation of λ is exactly REML estimation of the random-effect variance
- GAMs and GLMMs are **two sides of the same coin**

What this means in practice

- GAMs can be fitted using GLMM software
- The smoothing parameter λ is determined by the random-effect variance: $\sigma_b^2 = \sigma^2/\lambda$
- REML estimation of λ is exactly REML estimation of the random-effect variance
- GAMs and GLMMs are **two sides of the same coin**

A GAM is a GLMM with specially structured random effects

GAMs in glmmTMB

Since the GAM smooth is a GLMM, it can be fitted in glmmTMB using mgcv smooth terms:

```
library(glmmTMB)
model <- glmmTMB(y ~ s(x), family = poisson, data = dat)
```

GAMs in glmmTMB

Since the GAM smooth is a GLMM, it can be fitted in glmmTMB using mgcv smooth terms:

```
library(glmmTMB)
model <- glmmTMB(y ~ s(x), family = poisson, data = dat)
```

You can combine GAM smooths with random effects in the same model:

```
model <- glmmTMB(y ~ s(x) + (1|group),
                 family = poisson, data = dat)
```

GAMs in glmmTMB

Since the GAM smooth is a GLMM, it can be fitted in glmmTMB using mgcv smooth terms:

```
library(glmmTMB)
model <- glmmTMB(y ~ s(x), family = poisson, data = dat)
```

You can combine GAM smooths with random effects in the same model:

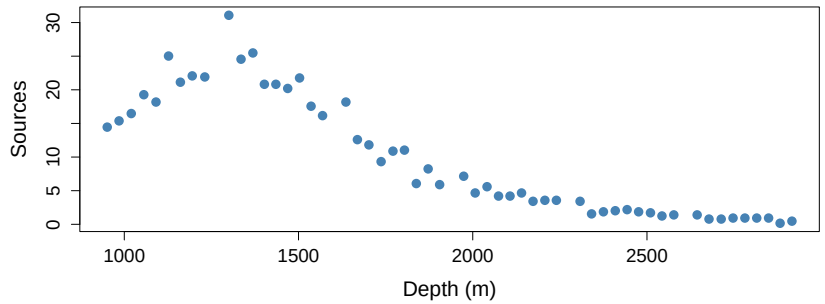
```
model <- glmmTMB(y ~ s(x) + (1|group),
                 family = poisson, data = dat)
```

- Uses the Wood (2004, 2011) diagonalization internally
- Supports all mgcv basis types

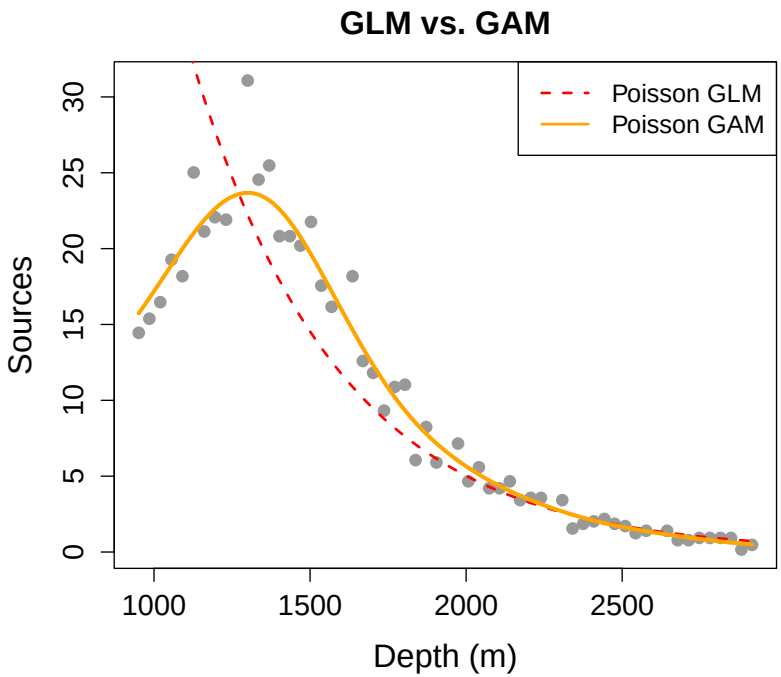
Example: bioluminescence at depth

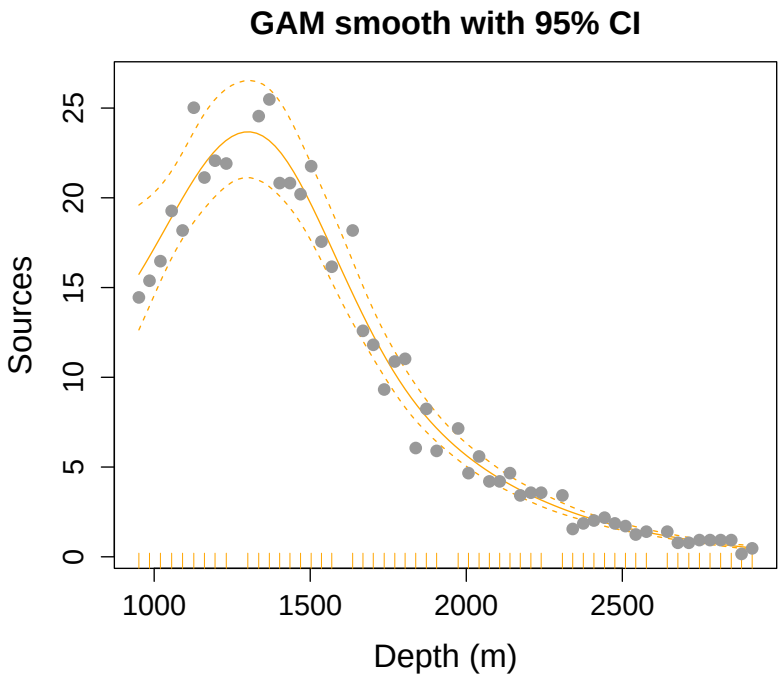
Data from Gillibrand et al. (2007), via Zuur et al. (2009)

- 1,654 observations across 19 stations
- Response: number of bioluminescent sources
- Predictor: sample depth (m)
- Strong non-linear relationship with depth



ISIT: GLM vs. GAM





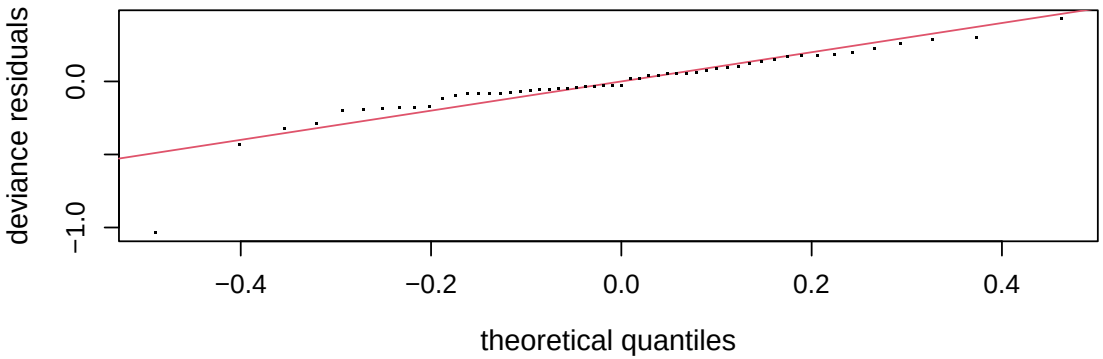
ISIT: fitting

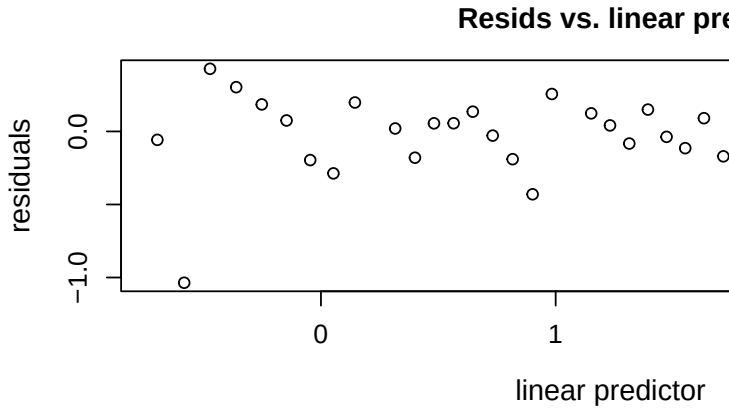
```
library(mgcv)
ISIT <- read.csv("../data/ISIT.csv")
dat_ex <- subset(ISIT, Station == 9)

model_gam <- gam(Sources ~ s(SampleDepth, bs="cr"),
                 family = Gamma(link="log"), data = dat_ex,
                 method = "REML")
```

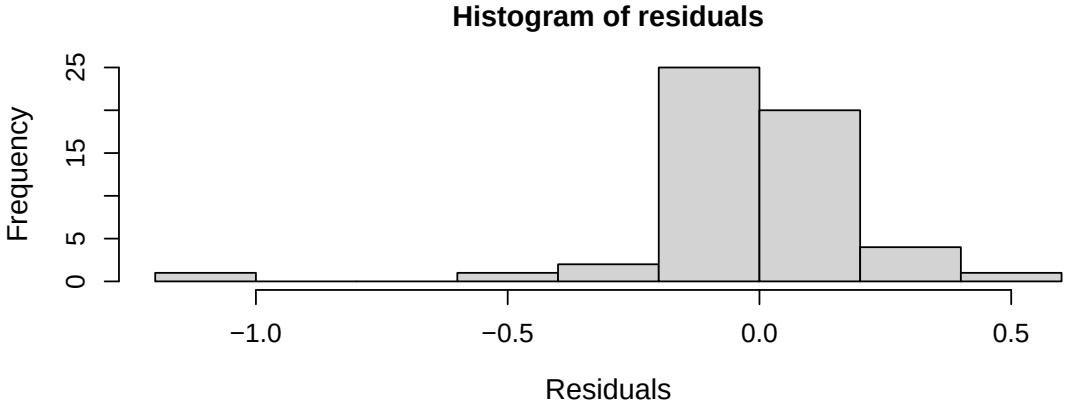
ISIT: summary

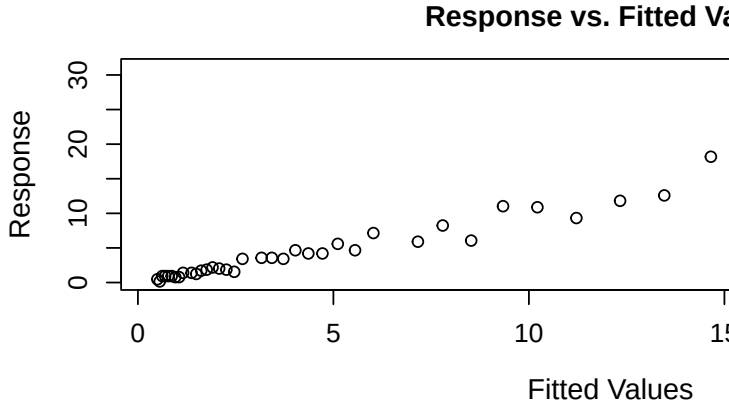
```
##
## Family: Gamma
## Link function: log
##
## Formula:
## Sources ~ s(SampleDepth, bs = "cr")
##
## Parametric coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  1.70174    0.02749    61.9   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##                  edf Ref.df     F p-value
## s(SampleDepth) 5.358  6.472 310.6  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

ISIT: model checking









ISIT: multiple stations

```
model_full <- gam(Sources ~ s(SampleDepth, bs="cr") +
                    s(Station, bs="re"),
                  family = Gamma(link="log"), data = ISIT,
                  method = "REML")
```

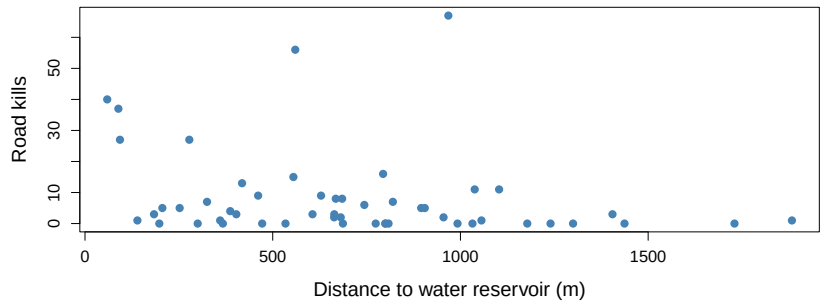
- `s(Station, bs="re")`: random intercept for each station — a GAMM
- The same in `glmmTMB`:

```
library(glmmTMB)
model_tmb <- glmmTMB(Sources ~ s(SampleDepth, bs="cr") +
                       (1|Station),
                     family = Gamma(link="log"), data = ISIT)
```

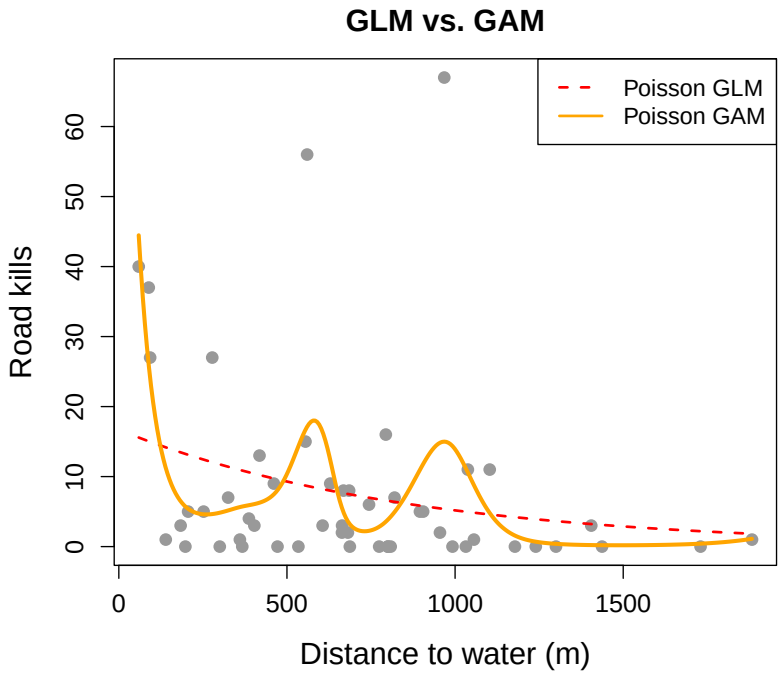
Road-killed toads in Portugal

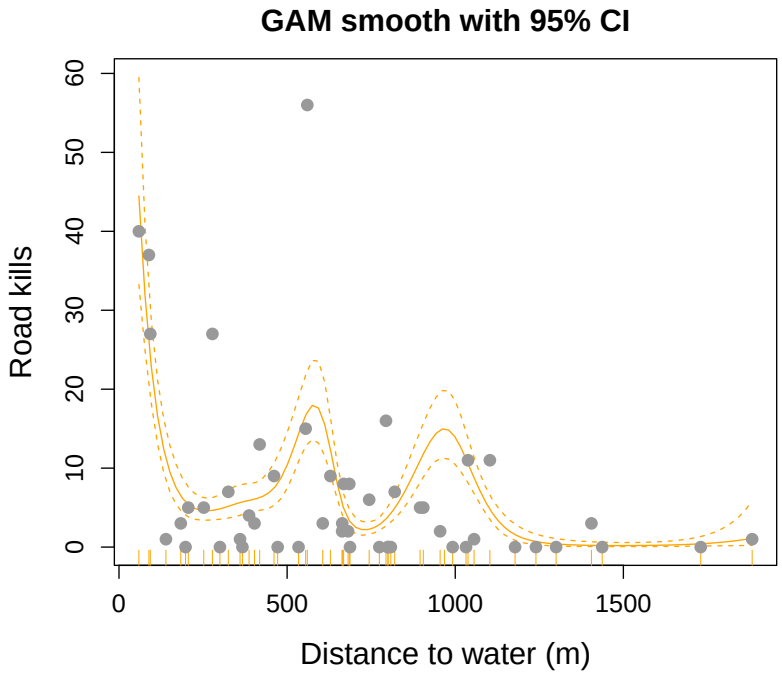
Data from Zuur et al. (2009), originally Mira et al.

- 52 road sectors in Portugal
- Response: number of road-killed *Bufo calamita* (natterjack toad)
- Predictor: distance to nearest water reservoir (`D.WAT.RES`)
- Non-linear relationship expected — toads concentrate near water



Road kills: GLM vs. GAM





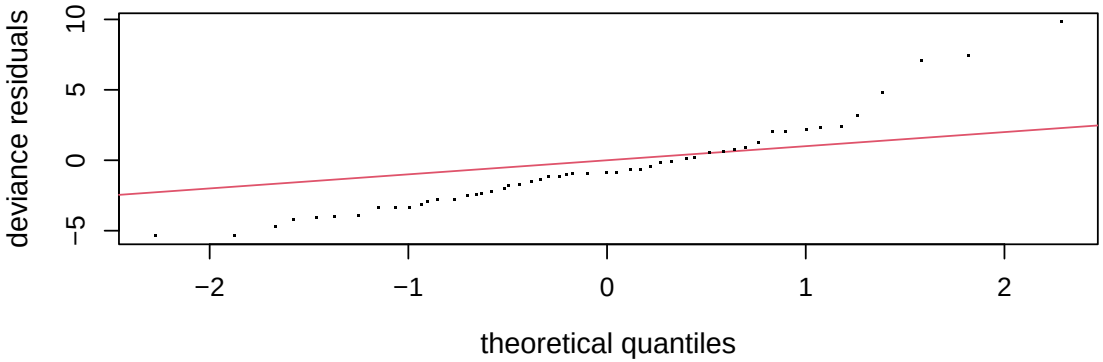
Road kills: fitting

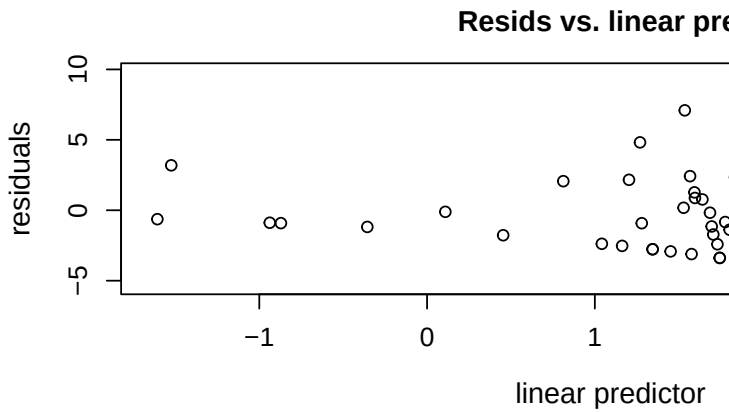
```
rk <- read.csv("../data/RoadKills.csv")

model_rk <- gam(BufoCalamita ~ s(D.WAT.RES, bs="cr"),
                family = poisson, data = rk, method = "REML")
```

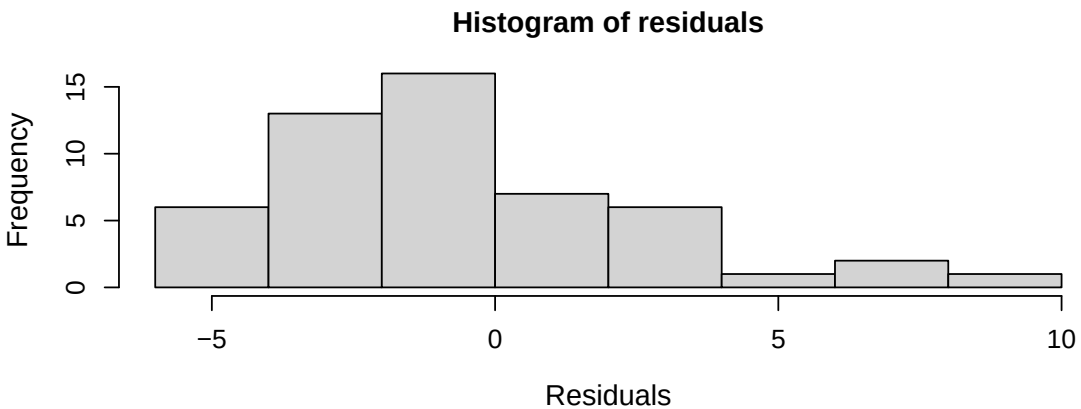
Road kills: summary

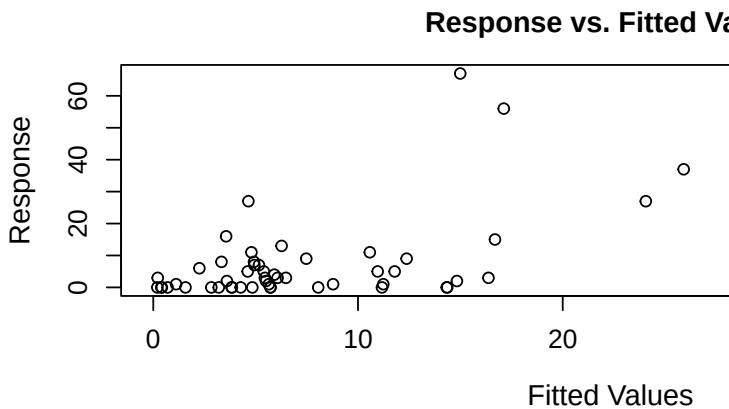
```
##
## Family: poisson
## Link function: log
##
## Formula:
## BufoCalamita ~ s(D.WAT.RES, bs = "cr")
##
## Parametric coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)  1.63580    0.08485   19.28   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##                edf Ref.df Chi.sq p-value
## s(D.WAT.RES) 8.598  8.932  260.1  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Road kills: model checking



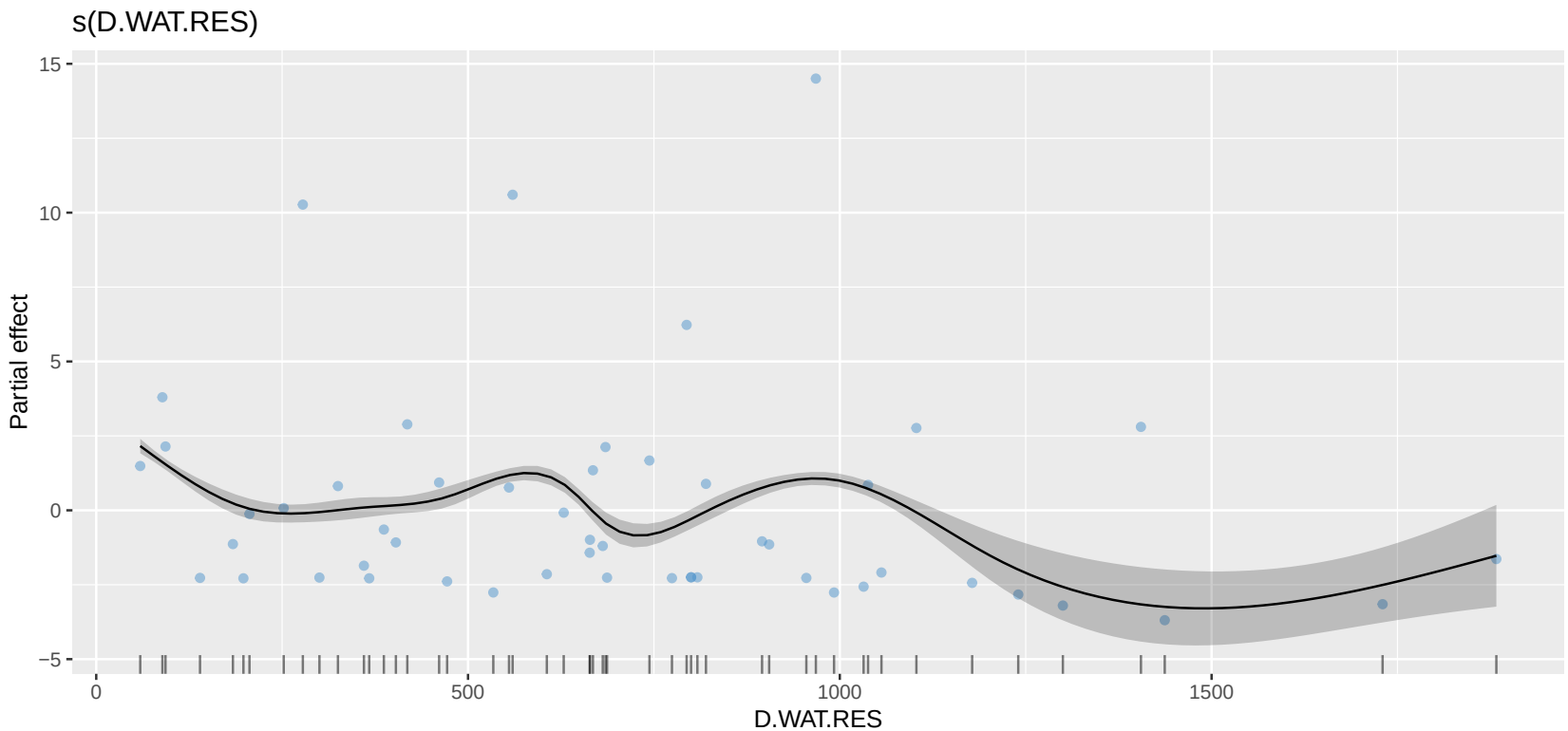






Road kills: visualisation with `gratia`

```
library(gratia)
draw(model_rk, residuals=TRUE)
```



Road kills: adding more predictors

```
model_rk2 <- gam(BufoCalamita ~ s(D.WAT.RES, bs="cr") +
                   s(D.WAT.COUR, bs="cr") +
                   s(L.D.ROAD, bs="cr"),
                 family = poisson, data = rk, method = "REML")
```

- Multiple smooths can be combined additively
- Each smooth's edf is estimated and penalized independently
- Use AIC or REML score to compare models

Summary

- **Ridge regression**: penalizes $\|\beta\|^2$ — equivalent to a normal prior; conditional mode = MAP estimate (not a BLUP)
- **GAMs** extend GLMs with smooth functions $f(x)$ instead of linear terms $x\beta$
- **Splines** represent $f(x)$ as weighted basis functions; many types available
- **Wiggliness penalty** controls smoothness; λ is estimated automatically
- **REML** is the recommended criterion for smoothing parameter selection
- **GAMs are GLMMs**: the penalized smooth decomposes into fixed + random effects

End